

CHUHAN WANG

+1 213-284-8616 | chuhanw@usc.edu | [LinkedIn](#) | [GitHub](#) | Los Angeles, CA

EDUCATION

University of Southern California — Los Angeles, CA | September 2025 – December 2027

M.S. in Computer Science

University of Waterloo — Waterloo, Canada | September 2022 – June 2025

B.E.S. in Geomatics, Minor in Computing | GPA: 3.91/4.0 (Top Standing)

TECHNICAL SKILLS

Languages: Java, Python, C++, TypeScript/JavaScript, SQL

Frameworks & APIs: Spring Boot, Flask, Fastify, React, REST APIs, Microservices

AI & Agent Development: LangChain, MCP, LLM API Integration, Prompt Engineering, Agentic Workflows, Claude Code, Cursor

Databases & Messaging: PostgreSQL, MySQL, MongoDB, Redis, Kafka, BullMQ

Cloud & DevOps: AWS (EC2, S3, EKS), Docker, Kubernetes, CI/CD, GitHub Actions, Git, Linux

INTERNSHIPS

Baidu — AI Product Manager Intern, Miaoda AI App Builder | Beijing, China

June 2026 – September 2026

- Led the 0-to-1 launch of Miaoda paid template marketplace, defining 100–10K credit pricing and an 80/20 creator revenue split.
- Wired end-to-end transaction workflows for the marketplace launch, supporting a full 191K+ app ecosystem across the entire platform creator base.
- Designed anti-abuse and transaction-safety controls spanning 20 transaction, failure, retry, and concurrency scenarios for the marketplace launch.
- Deployed mass-copying detection flagging anomalous copy attempts daily, automatically suppressing associated creator payouts on flagged accounts.
- Specified **tracedId**-based idempotency, distributed locking, rollback, and reconciliation logic to guarantee transaction consistency at full scale.
- Redesigned marketplace ranking around 7-day behavioral signals, separating general app discovery from conversion-oriented template ranking.
- Reduced Top-20/50 weekly overlap from 91% to 79% through the ranking redesign, surfacing a much wider variety of apps in top placement slots weekly.
- Diagnosed new-app overexposure during A/B rollout, tracing user-reported issues to over-aggressive freshness boosting.
- Recalibrated the 72h-to-60h decay window with minimum-sample thresholds, restoring quality metrics fully to baseline once the regression was traced.
- Improved template conversion while reducing manual ranking interventions from daily adjustments to weekly tuning.

MusicNBrain — Software Engineering Intern | San Francisco, CA (Remote)

May 2026 – Present

- Built a **Python** and **FastAPI** orchestration backend integrating Avatar and Meeting Bot services through **MCP**, **REST APIs**, and **WebSocket** connections.
- Managed a 9-state AI lesson lifecycle within the backend, adding health monitoring and fault-tolerant cleanup for every single session interruption.
- Re-architected the backend into per-session isolated execution, separating bots and subprocesses to fully support up to 8 concurrent live lessons.
- Shipped the real-time Avatar service on **AWS EC2**, enabling 16-kHz **WebSocket** audio streaming and remote model inference for every live lesson held.
- Formed an **MCP**-based avatar-training pipeline with automated video validation and retry handling, cutting failed training workflows by a full 45%.

PROJECTS

AIGC E-commerce Video Generation | ByteDance Hackathon — Outstanding Team Award | React, TypeScript, Fastify, PostgreSQL, Redis, BullMQ

- Constructed a four-factor creative feature framework spanning product category, conversion objective, target audience, and selling strategy used.
- Developed a recommendation algorithm ranking audience-strategy combinations by **ROAS** and **GMV**, guiding subsequent video creation decisions made.
- Applied empirical Bayes shrinkage to reduce small-sample bias, implementing tunable multi-objective scoring across efficiency and revenue metrics.
- Devised a human-in-the-loop AIGC pipeline with artifact validation, async **BullMQ** jobs, failure recovery, and **PostgreSQL**-backed generation metadata.

Agent AI: Retrieval-Augmented Document Q&A System | React, Node.js, Express, LangChain, OpenAI, Chroma, PostgreSQL, Redis, Docker

- Separated ingestion from retrieval via a **Redis**-backed **BullMQ** pipeline, chunking and embedding PDFs once on upload with full job state persistence.
- Slashed per-query cost from $O(n)$ to $O(1)$ through the ingestion-retrieval split, adding retries and crash recovery to the upload pipeline itself.
- Configured storage using **PostgreSQL** for metadata and **Chroma** for vector collections, enabling token-level streaming via **SSE** for every user response.
- Cut time-to-first-token to roughly 1.2 seconds through the **SSE** streaming layer, keeping perceived latency low across every long text generation.
- Coded evaluation and benchmarking scripts measuring retrieval accuracy near 90% and P50/P95 latency, isolating **LLM** inference as the main bottleneck.
- Containerized the full stack with **Docker Compose** for one-command local deployment, matching the exact setup used across production environments.